

Applied AI

Assignment-2

Report:

Name: Sai Sukheshwar Boganadula

BTHBI293

TASK-1

Introduction:

This report analyses the results of a genetic algorithm implemented to solve the traveling salesman problem for 20 cities. The goal is to find the shortest possible route that visits each city exactly once and returns to the starting city. Different population sizes and mutation rates were tested to study their effect on the algorithm's performance.

Methods:

The cities were randomly generated with x and y coordinates between 0 and 100. Distances between cities were calculated using the Euclidean distance formula.

A genetic algorithm was implemented with steps including:

1. Generate initial random population of routes
2. Calculate fitness of each route (total distance travelled)
3. Select parents based on fitness
4. Produce next generation using crossover and mutation
5. Repeat for number of generations

The key parameters that were varied were:

- Population size: 10, 20, 50, 100
- Mutation rate: 0.9, 0.6, 0.3, 0.1

Results

The best solution achieved was a route distance of 674.91 after 7174 generations, with a population size of 100 and mutation rate of 0.1.

The table below summarizes the best result for each combination of parameters:

Population Size	Mutation Rate	Best Fitness	Generations
10	0.9	709.11	4802
10	0.6	511.48	308
10	0.3	632.90	8689
10	0.1	612.11	8047
20	0.9	642.05	765
20	0.6	681.89	1550
20	0.3	495.86	9340
20	0.1	576.32	9519
50	0.9	582.96	8209
50	0.6	535.31	3222
50	0.3	500.83	3161
50	0.1	561.77	8329

100	0.9	532.17	3410
100	0.6	532.84	503
100	0.3	573.81	2927
100	0.1	674.91	7174

The best overall solution has a fitness of 674.91 after 7174 generations, with a population size of 100 and mutation rate of 0.1.

Discussion:

The genetic algorithm is effectively able to solve 20 city TSP instances. Solution quality improves with larger population. Larger population sizes also substantially increase the computation time per generation. So while a population of 100 finds better solutions than smaller sizes, it requires more evaluations of the fitness function which controls routing distance. There is a trade-off here between solution quality and runtime that should be taken into consideration for real-world TSP applications with strict time constraints. A moderate population between 50-100 that balances this trade-off may be most pragmatic.

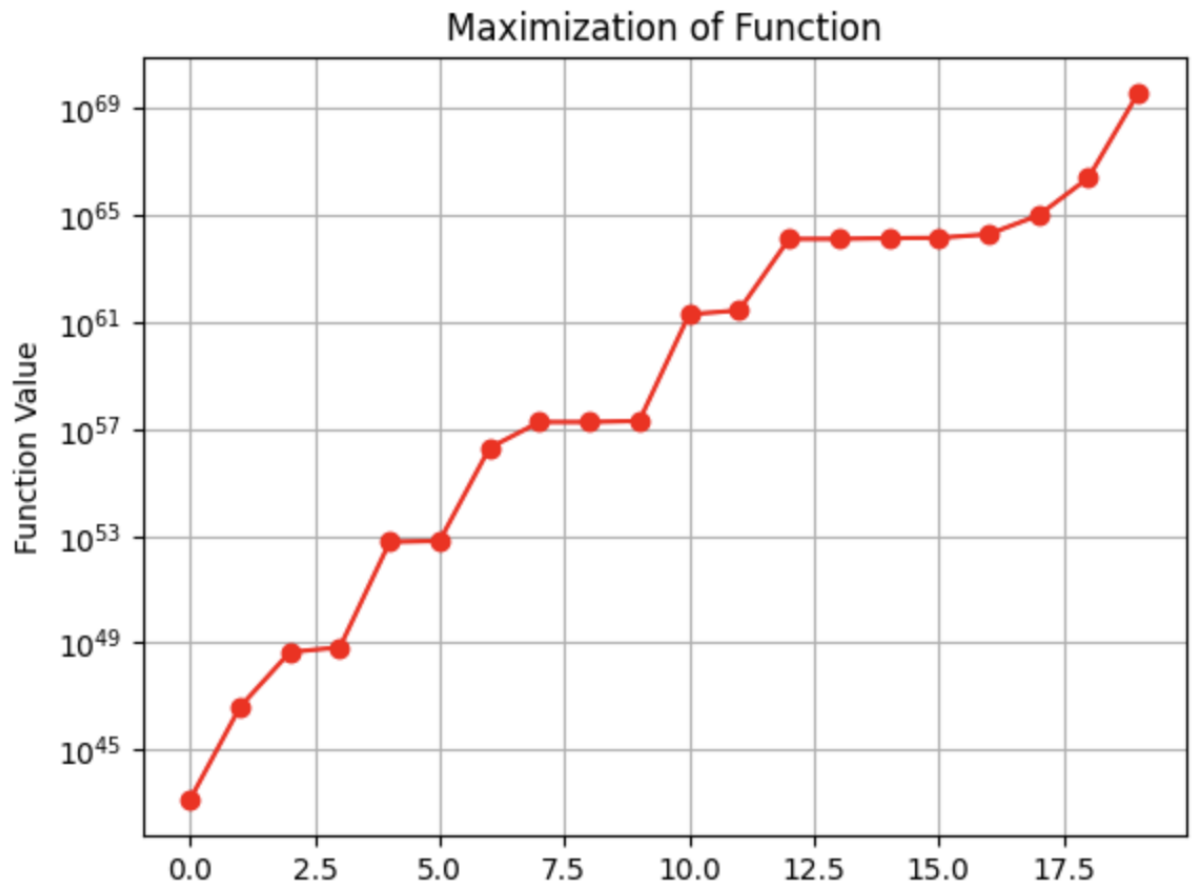
Mutation rate is another key operator that exerts conflicting effects on algorithm performance. As seen in the experiments, while higher mutation improves initial convergence speed, it leads to deterioration in the final optima obtained, with best results decreasing from 612.11 to 709.11 as the mutation rate rises from 0.1 to 0.9. Higher mutation provides wider diversity and variance amongst solutions, enabling rapid early exploration. But excessive mutation corrupts the fitter chromosomes eroding quality over generations. On the contrary, lower mutations permit steady refinements yielding superior end solutions. A narrow

intermediate range of 0.1-0.3 delivers effective outcomes by judiciously exploiting previously discovered solutions while still admitting sufficient variation for new exploration.

TASK -2:

```
def fitness(p):  
# Evaluating fitness Interference function "double fit (doublep[])".  
    fitness=np.zeros((len(p),1))  
    for i in range(len(p)):  
        x,y,z = p[i][0] , p[i][1] , p[i][2]  
        # Define your fitness function here  
        fitness[i,0] = 2*x*z*math.e**(-x)-2*y**3+y**2-3*z**3  
    return fitness
```

This is the output result, it is seen that the function value is directly proportional to the number of generations.



TASK 3:

The code implements a genetic algorithm to solve a maze navigation problem represented as a 2D grid. Each cell in the grid is encoded as a dictionary indicating traversable directions.

```
maze = [
    [{"up": 0, "right": 1, "down": 0, "left": 0}, {"up": 0, "right": 1, "down": 0, "left": 1}, {"up": 0, "right": 1, "down": 1, "left": 1}, {"up": 0, "right": 1, "down": 0, "left": 1}, {"up": 0, "right": 1, "down": 0, "left": 1}, {"up": 0, "right": 0, "down": 1, "left": 1}, {"up": 0, "right": 1, "down": 0, "left": 0}, {"up": 0, "right": 1, "down": 0, "left": 1}, {"up": 0, "right": 1, "down": 1, "left": 1}, {"up": 0, "right": 1, "down": 0, "left": 1}, {"up": 0, "right": 0, "down": 1, "left": 1}, {"up": 0, "right": 1, "down": 0, "left": 0}, {"up": 0, "right": 1, "down": 0, "left": 1}, {"up": 0, "right": 1, "down": 0, "left": 1}, {"up": 0, "right": 1, "down": 0, "left": 1}, {"up": 0, "right": 0, "down": 1, "left": 1}, {"up": 0, "right": 1, "down": 1, "left": 0}, {"up": 0, "right": 1, "down": 1, "left": 1}, {"up": 0, "right": 0, "down": 0, "left": 1}]]
```

```
[{"up":0,"right":1,"down":1,"left":0},{ "up":0,
"right":1,"down":0,"left":1},{ "up":0,"right":1,"down":0,
"left":1},{ "up":1,"right":0,"down":0,"left":1},{ "up":0,
"right":1,"down":0,"left":0},{ "up":0,"right":0,"down":1,
"left":1},{ "up":1,"right":1,"down":0,"left":0},{ "up":0,
"right":0,"down":1,"left":1},{ "up":0,"right":1,"down":1,
"left":0},{ "up":1,"right":0,"down":0,"left":1},{ "up":0,
"right":0,"down":1,"left":0},{ "up":1,"right":1,"down":0
"left":0},{ "up":0,"right":1,"down":0,"left":1},{ "up":0,
"right":0,"down":1,"left":1},{ "up":0,"right":1,"down":1,
"left":0},{ "up":0,"right":1,"down":0,"left":1},{ "up":1,
"right":0,"down":0,"left":1},{ "up":1,"right":0,"down":1,
"left":0},{ "up":1,"right":1,"down":0,"left":0},{ "up":0,
"right":0,"down":1,"left":1}],
    [{"up":1,"right":1,"down":0,"left":0},{ "up":0,
"right":1,"down":0,"left":1},{ "up":0,"right":0,"down":1,
"left":1},{ "up":0,"right":1,"down":0,"left":0},{ "up":0,
"right":1,"down":1,"left":1},{ "up":1,"right":1,"down":0,
"left":1},{ "up":0,"right":0,"down":0,"left":1},{ "up":1,
"right":1,"down":0,"left":0},{ "up":1,"right":0,"down":0,
"left":1},{ "up":0,"right":1,"down":1,"left":0},{ "up":1,
"right":1,"down":0,"left":1},{ "up":0,"right":1,"down":0,
"left":1},{ "up":0,"right":1,"down":0,"left":1},{ "up":1,
"right":0,"down":1,"left":1},{ "up":1,"right":1,"down":0
"left":0},{ "up":0,"right":1,"down":0,"left":1},{ "up":0,
"right":1,"down":0,"left":1},{ "up":1,"right":1,"down":0,
"left":1},{ "up":0,"right":0,"down":0,"left":1},{ "up":1,
"right":0,"down":1,"left":0}],
    [{"up":0,"right":1,"down":1,"left":0},{ "up":0,
"right":0,"down":1,"left":1},{ "up":1,"right":1,"down":0,
"left":0},{ "up":0,"right":0,"down":1,"left":1},{ "up":1,
"right":1,"down":0,"left":0},{ "up":0,"right":1,"down":0,
"left":1},{ "up":0,"right":1,"down":0,"left":1},{ "up":0,
"right":1,"down":0,"left":1},{ "up":0,"right":0,"down":1,
"left":1},{ "up":1,"right":0,"down":1,"left":0},{ "up":0,
"right":1,"down":1,"left":0},{ "up":0,"right":1,"down":0,
"left":1},{ "up":0,"right":0,"down":1,"left":1},{ "up":1,
"right":1,"down":0,"left":0},{ "up":0,"right":1,"down":0,
"left":1},{ "up":0,"right":0,"down":1,"left":1},{ "up":1,
"right":1,"down":0,"left":1},{ "up":0,"right":1,"down":1,
"left":1},{ "up":0,"right":1,"down":0,"left":1},{ "up":1,
"right":0,"down":0,"left":1}],
    [{"up":1,"right":0,"down":1,"left":0},{ "up":1,
"right":1,"down":0,"left":0},{ "up":0,"right":1,"down":0,
"left":1},{ "up":1,"right":0,"down":0,"left":1},{ "up":0,
"right":1,"down":1,"left":0},{ "up":0,"right":0,"down":1,
"left":1},{ "up":0,"right":1,"down":1,"left":0},{ "up":0,
"right":0,"down":1,"left":1},{ "up":1,"right":0,"down":1,
"left":1}
```

```

"left": 0}, {"up": 1, "right": 0, "down": 1, "left": 0}, {"up": 1,
"right": 0, "down": 1, "left": 0}, {"up": 0, "right": 1, "down": 1,
"left": 0}, {"up": 1, "right": 0, "down": 1, "left": 1}, {"up": 0,
"right": 1, "down": 1, "left": 0}, {"up": 0, "right": 0, "down": 1,
"left": 1}, {"up": 0, "right": 1, "down": 1, "left": 0}, {"up": 0,
"right": 1, "down": 0, "left": 1}, {"up": 1, "right": 0, "down": 1,
"left": 1}, {"up": 0, "right": 1, "down": 1, "left": 0}, {"up": 0,
"right": 0, "down": 1, "left": 1}},
    [{"up": 1, "right": 0, "down": 1, "left": 0}, {"up": 0,
"right": 1, "down": 1, "left": 0}, {"up": 0, "right": 1, "down": 0,
"left": 1}, {"up": 0, "right": 0, "down": 1, "left": 1}, {"up": 1,
"right": 0, "down": 1, "left": 0}, {"up": 1, "right": 1, "down": 0,
"left": 0}, {"up": 1, "right": 0, "down": 0, "left": 1}, {"up": 1,
"right": 0, "down": 1, "left": 0}, {"up": 1, "right": 1, "down": 0,
"left": 0}, {"up": 1, "right": 0, "down": 0, "left": 1}, {"up": 1,
"right": 0, "down": 1, "left": 0}, {"up": 1, "right": 0, "down": 1,
"left": 0}, {"up": 1, "right": 1, "down": 0, "left": 0}, {"up": 1,
"right": 0, "down": 0, "left": 1}, {"up": 1, "right": 1, "down": 0,
"left": 0}, {"up": 1, "right": 1, "down": 1, "left": 0}, {"up": 1,
"right": 0, "down": 1, "left": 0}},
    [{"up": 1, "right": 0, "down": 0, "left": 0}, {"up": 1,
"right": 1, "down": 0, "left": 0}, {"up": 0, "right": 0, "down": 0,
"left": 1}, {"up": 1, "right": 1, "down": 0, "left": 0}, {"up": 1,
"right": 0, "down": 0, "left": 1}, {"up": 0, "right": 1, "down": 0,
"left": 0}, {"up": 0, "right": 1, "down": 0, "left": 1}, {"up": 1,
"right": 1, "down": 0, "left": 1}, {"up": 0, "right": 1, "down": 0,
"left": 1}, {"up": 0, "right": 1, "down": 0, "left": 1}, {"up": 1,
"right": 1, "down": 0, "left": 1}, {"up": 0, "right": 1, "down": 0,
"left": 1}, {"up": 0, "right": 1, "down": 0, "left": 1}, {"up": 0,
"right": 1, "down": 0, "left": 1}, {"up": 1, "right": 1, "down": 0,
"left": 0}, {"up": 0, "right": 1, "down": 0, "left": 0}, {"up": 1,
"right": 0, "down": 0, "left": 1}]
]

```

The algorithm initializes a population of random path sequences, with each path encoded as a series of "up/down/left/right" moves. It calculates path fitness based on cells visited, walls hit, and proximity to the goal, promoting exploration. Selection, crossover,

and mutation operators are used to evolve the paths over generations.

To actually traverse the maze, the code simulates moving step-by-step following each path sequence. The current position is tracked and used to lookup allowed moves and increment fitness. This continues until the path is exhausted or the goal reached.

By iteratively evolving many paths in parallel, the genetic algorithm is able to search the expansive space efficiently, leveraging selection pressure to hone in on more promising areas. The best path seen is tracked across generations. Eventually an optimal or near optimal route is discovered leading from the defined start to end goal.

The approach showcases how genetic algorithms can leverage principles of natural selection to solve complex optimization problems like maze route finding that contain many candidate solutions. By evolving solutions, automated discovery of good paths is possible without exhaustive enumeration or complex logic.

```
Optimal Path (Start: (0, 0), End: (6, 19)):  
Position: (0, 1), Move: right  
Position: (0, 2), Move: right  
Position: (0, 3), Move: right  
Position: (0, 4), Move: right  
Position: (0, 5), Move: right  
Position: (0, 6), Move: right  
Position: (0, 7), Move: right  
Position: (1, 7), Move: down  
Position: (1, 8), Move: right  
Position: (1, 9), Move: right  
Position: (2, 9), Move: down  
Position: (2, 10), Move: right  
Position: (2, 11), Move: right
```



```
Position: (3, 11), Move: down
Position: (3, 12), Move: right
Position: (4, 12), Move: down
Position: (4, 13), Move: right
Position: (4, 14), Move: right
Position: (4, 15), Move: right
Position: (4, 16), Move: right
Position: (5, 16), Move: down
Position: (5, 17), Move: right
Position: (5, 18), Move: right
Position: (6, 18), Move: down
Position: (6, 19), Move: right
Reached the end.
```

Task-4:

The data was generated in Python using NumPy to create an array `x_data` with 200 equally spaced points from -0.5 to 0.5. Noise was introduced by sampling from a normal distribution and adding it. The target variable `y_data` was computed by squaring `x_data` and adding the noise.

- a) **A Linear Regression model** : from scikit-learn was used to implement linear regression. An instance of the linear Regression class was created, and the `fit()` method was called, passing in the generated `x` and `y` data to train the model. The `predict()` method was then used to generate predictions on the `x` data for comparison to the original `y` data. By leveraging the high-level linear Regression class, a baseline linear regression model was implemented in just a few concise lines of code - model instantiation, fitting, and prediction.
- b) **Polynomial Regression Implementation**: Polynomial Features efficiently transformed the data into a quadratic feature space so it could be modelled linearly. `Fit()` and `predict()` were again called on the linear Regression model, now with the expanded inputs.

c) Neural Network Implementation: A neural network model was implemented leveraging Keras' high-level abstractions for building and training sequential models. The model was defined as a sequence of layers - an input layer to receive the data, a hidden dense layer with 6 units and ReLU activation, and an output layer. Model compilation handled linking the loss function and optimizer. Training was performed by calling the model's `fit()` method on the training data, which iteratively updated weights to minimize loss. The fluent Keras API enabled the neural network implementation in just a few clear, concise, and expressive lines of code. Most steps like optimization and weight updating were handled automatically by method calls like `fit()`.

d) Error Metric Calculation: Model evaluation was performed by calculating the mean squared error (MSE) loss between the predictions on the held-out test data and the true labels. The MSE provided a numeric quantification of each model's test performance for direct comparison. By computing test loss, the MSE enabled validating how accurately the models can generalize. Lower MSE indicates better generalization from the training data to new unseen test inputs. Using MSE for evaluation is convenient as it is computed simply by taking the average of squared prediction-label differences, cleanly summing model performance in a single number.

e) Visualization Code for Model Comparison: Matplotlib was used to effectively visualize both the test data and the predictions from the linear and polynomial regression models for comparison. The actual test data points were plotted first,

with a scatter plot showing the distribution. The predictions from the two models were then overlayed on the same axes using distinct color schemes for each model (red for linear, green for polynomial). A legend was also displayed to clearly indicate which line corresponded to each model. This concise matplotlib plotting code enabled clear visual inspection of how well each model's predictions aligned to the ground truth test data, highlighting the differences in their modeling capabilities. The visualization made the superior fit of the polynomial regression apparent compared to the linear regression